

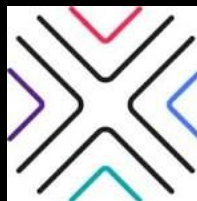


Spec-Driven Development for Airflow DAGs



Kyle McCluskey

About Me



ASTRONOMER Product Customers Docs Resources Airflow Pricing Log In Book a Demo Get Started Free

MEET THE CHAMPIONS

 Siva Krishna Yetukuri Cloud Data Architect Wynn Las Vegas Las Vegas	 Venkatesh Wenz Senior Software Engineer JPMorgan Chase Paraná, Brazil	 Dmitri Zverev Software Engineer Foursquare Belgrade, Serbia	 Shivani Chauhan Data Engineer II Petco Ranchi, India	 Ryan Delgado Staff Software Engineer, Data Platform Ramp NYC, USA	 Dave Tuazon Data Engineering Manager Northern Trust Bacoor City, Philippines
 Elder Enekeke	 Raviteja Tholupunoori	 Bhavani Ravi	 Ross Lovellace	 Victor Iwueha	 Kyle McCluskey

About TXI



About TXI

TXI builds **AI-powered workflows**
for industrial companies



Modernizing
operations



Connecting
data



Accelerating
decisions

abbvie

GATX

AITX

MCAA

Motorcity Systems

DICKSON

Northwestern

AMA

AccuWeather

GitHub



Spec-Driven Development

SDD @ TXI



- Rigorous response to “vibe-coding”
- OpenSpec, SpecKit, Superpowers
- Refine requirements with the coding assistant
- Generate tests first from spec
- Dispatch specific agents to implement and review

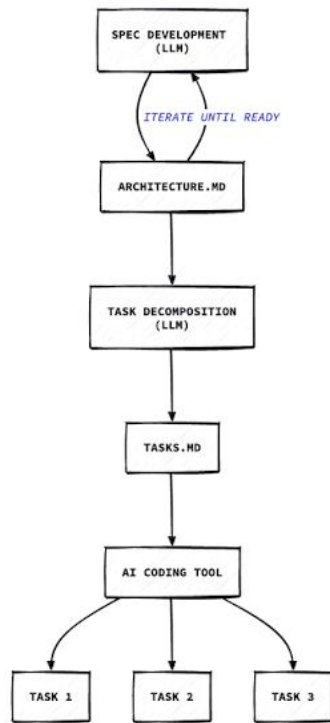
CCM / SKILLS

S Superpowers
founderjourney/claude-skills

136 installs 10 stars

SUMMARY

This is a development framework that stops Claude from immediately writing code and instead guides it through a structured approach: planning, then implementation, then review. It's designed for situations where thoughtful architecture matters more than speed, like multi-file refactors, TDD workflows, or systematic debugging sessions. The creator calls out the "write code immediately" antipattern directly, which feels like wisdom earned from watching AI agents make messes. With 136 installs and clean security audits, it's being used in the wild. If you've ever wished Claude would slow down and think before touching your codebase, this enforces exactly that discipline.





Data Engineering Workflows

Data Engineering w/ SDD



- “Very Stable” end of the Spec-Stability Spectrum
- Prescriptive declarations for Source and Target
- Deterministic
- Astronomer DAG factory

A screenshot of the DAG Factory documentation website. The header is dark blue with the "DAG Factory" logo, version "1.1.0 latest", a search bar, and a GitHub repository link for "astronomer/dag-factory". The main content area is white and features a "Home" link, the title "DAG Factory documentation", a brief description, a "Getting started" section with a list of links, and a right-hand sidebar with a "Table of contents" and various links like "Getting help" and "Contributing".

Home

DAG Factory documentation

Everything you need to know about how to build Apache Airflow® workflows using YAML files.

Getting started

Are you new to DAG Factory? This is the place to start!

- DAG Factory at a glance
 - Quickstart with Airflow standalone
 - Quickstart with Astro CLI
- Using YAML instead of Python
 - Traditional Airflow Operators
 - TaskFlow API

Table of contents

- Getting started
- Command Line
- Configuration
- Features
 - Dag-Factory 1.0 Released
 - Dag-Factory Supports Airflow 3
- Getting help
- Contributing
- License
- Privacy Notice
- Security Policy

- > I am working on a prototype of our full Airflow Ingestion Pipeline that leverages the Astronomer DAG Factory open-source tool. I want to develop a specification for what entities the pipeline needs to ingest, in what order and turn that specification into a DAGFactory YAML specification.
- I'll start by invoking the brainstorming skill, since we're designing a specification for new pipeline work — exploring requirements before jumping to YAML is exactly the right approach here.
 - **Skill**(superpowers:brainstorming)
 - | Successfully loaded skill
 - **Skill**(superpowers:brainstorming)
 - | Successfully loaded skill
 - I'm going to follow the brainstorming process. Let me start by exploring the project context — particularly the existing pipeline structure, since the CLAUDE.md mentions a pipeline ingestion flow and there's a `db_pipeline` working directory.

- I've got a clear picture of the current state. Before I ask questions, here's what I found that's directly relevant:
 - **The ingestion order already exists in code.** `run_all_ingestion.py` runs ~42 scripts sequentially, and `brc_ingestion_dag.py` already encodes the full dependency graph as a hand-written Airflow DAG with 7 tiers (cleanup → 16 independent reference tables → derived reference → entities → operational → events → details, plus a responsibility-matrix branch).
 - That existing DAG uses **fine-grained TaskFlow task groups** (extract → transform → validate → load) per entity, wired with Python `>>` operators.
 - **DAGFactory** (the Astronomer tool you want to use) is declarative: YAML defines tasks that reference Python callables/operators and dependencies. It supports `task_groups` and `dependencies` blocks.
- So this is really about **re-expressing an already-known ingestion graph as a DAGFactory YAML spec**, plus writing the entity/ordering spec that drives it. That reframes my questions. Let me ask them one at a time.

First, the most consequential one — scope:

□ Scope

For this prototype, how much of the ingestion graph should the DAGFactory YAML cover?

- > 1. **Full graph (all ~42 entities)**
Replicate the complete dependency graph from `brc_ingestion_dag.py` / `run_all_ingestion.py` — every reference, entity, operational, event, detail, and matrix table. Most faithful, larger YAML.
- 2. **Representative vertical slice**
A subset that exercises every tier and dependency pattern (one reference → entity → operational → event → detail chain + the matrix branch). Proves

• This is the key technical input. Classification is now unambiguous:

- **Single main() tasks** (no full E/T/V/L surface): `responsibility_types`, `cost_buckets`, `cost_bucket_user_teams`, `railcar_categories`, `shop_network`, `shopping_statuses`, `brc_statuses`, `responsibility_matrix_v2` (VL only – DAG already runs it as `main()`), and `compute_validation_flags`.
- **E/T/V/L task groups** (full ETVL): the remaining ~30 scripts, with two needing `has_transform=false` (`brc_qualifiers`, `railcar_class_codes`) and edge case (`railcar_lessee_history` shows ETV – no `load_to_target` – I'll verify during implementation).

Now let me propose the design. There's one genuinely open technical decision – **how DAGFactory passes data between the E/T/V/L phases**, since what Python `>>` + return values do. Let me present that as approaches with a recommendation, then the full design.

Proposed approaches: inter-phase data passing in DAGFactory

The problem: the Python DAG's phases return staging-file paths and pass them as args (`do_transform(extracted, ...)`). Pure YAML has no return plumbing.

Approach A – Deterministic staging paths + explicit YAML dependencies (recommended).
The existing helper `_staging_path(table, phase)` already derives the path purely from `(table, phase)`. So each phase callable can *recompute* it from its `op_kwargs` (`module_name`, `table_name`, flags) – it never needs the upstream return value to find its data. The only thing the return v was **ordering**, and in DAGFactory that comes from the explicit `dependencies:` block. No XCom templating at all. The phase logic lives in a small `dag_factory_callables.py` (thin wrappers mirroring `do_extract/do_transform/...`), and the YAML stays fully declarative.

Approach B – XCom path passing via Jinja in YAML. Each task pulls upstream output with `"{{ ti.xcom_pull(...) }}"` in `op_kwargs`. Faithful to t DAG's return-passing, but brittle, verbose, and error-prone in YAML.

I recommend **A** – it's the natural DAGFactory idiom and the deterministic-path helper already exists.

A second small choice: to avoid an $O(16 \times N)$ explosion of tier-to-tier edges in YAML, I propose inserting lightweight **tier-gate** `EmptyOperator` (`tier1_complete`, `tier2_complete`, ...). Each group depends on the prior gate and feeds the next. This preserves the Python DAG's "all of tier N of tier N+1" semantics with $O(N+M)$ readable edges instead of a cross-product.

Does Approach A + tier gates sound right before I lay out the full tier structure?

* Cooked for 3m 16s

> Yes I like approach A with the Tier Gates as that conforms to the DAGFactory idiom and avoids the tier-to-tier edges problem

>> accept edits on (shift+tab to cycle)



Airflow

DAGs

Cluster Activity

Datasets

Security

Browse

Admin

Docs



12:01 UTC

AU

DAG: brc_data_ingestion BRC pipeline: delete, reseed, and ingest all 28 tables from seed data

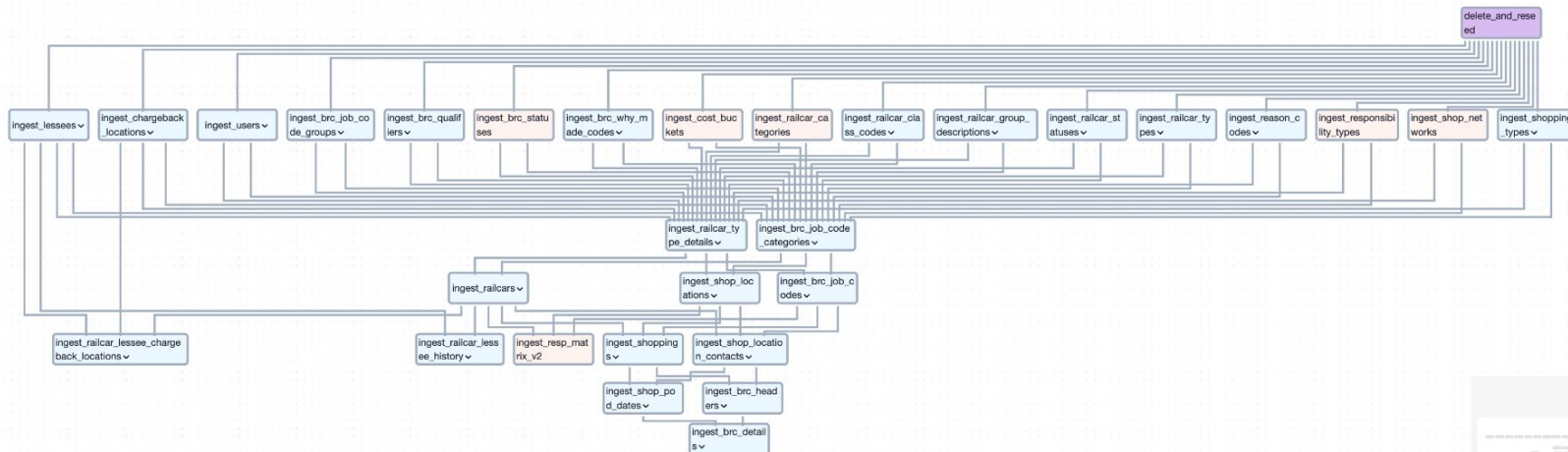
Schedule: None

Next Run ID: None



brc_data_ingestion

Details Graph Gantt Code Event Log Run Duration Task Duration Calendar

Layout:
Top -> Bottom


+

-



React Flow

Key Takeaways for DE



- Understanding your Data is the hard part
- Applicable to multiple DE technologies (DBT, Great Expectations)
- Most time spent refining requirements and planning
- Process is still rigorous TDD
- Code reviews are essential, but different
 - Not "Check that the thing was done right."
 - But rather "Check that the right thing was done."



Machine Learning Workflows

Machine Learning w/ SDD



- “Not Very Stable” end of the Spec-Stability Spectrum
- Qualitative Descriptions of User Behavior
- Non - Deterministic
- Astronomer Human-In-The-Loop skill

CCM / SKILLSSKILL



Airflow Hitl
astronomer/agents 

 645 installs  378 stars

SUMMARY

Adds approval gates, form inputs, and human-driven branching to Airflow DAGs using deferrable operators that release worker slots while waiting. You get `ApprovalOperator` for approve/reject flows, `HITLBranchOperator` for routing based on human choices, and `HITLEntryOperator` for pure form collection. The skill is built around a discover-then-code pattern: query the live Airflow registry for current constructor signatures before writing any tasks, since the provider changes params across releases. Responses come through the Airflow UI or REST API. Requires Airflow 3.1+. The safety checklist and external integration examples (Slack bots, custom scripts) are the most useful parts if you need humans in the loop without blocking workers.

AI generates results of varying confidence



Line Items

[Export](#)

[All](#) [Needs Review](#) [Waiting on Shop](#) [Approved](#) [Rejected](#) | [All \(46\)](#) [Owner \(1\)](#) [Lessee \(45\)](#)

☐	Item # ↑↓	Status ↑↓	Loc	Qty	Job Code ↑↓	QA	WMC	Description	Labor	Material	Total ↑↓	CB	CB Labor	CB Material	CB Total	Responsible ↑↓	CB Location	Confidence ↑↓	Confidence Reason	Internal N
☐	1	WAITING ON SHOP	—	1	4999	NX	25	CAR IS EQUIPPED WITH REFLECTIVE SHEETING	\$0.00	\$0.00	\$0.00	✓	\$0.00	\$0.00	\$0.00	9 - LESSEE	Rio Tinto Alcan Inc. (0007 37)	LOW	- More specific Job Code identified. Top 3: 5502, 5500, 8520 - Responsibility allocation was overridden	Allocation: It was overridden
☐	2	WAITING ON SHOP	—	1	4999	EU	25	CAR IS NOT EQUIPPED WITH CCSBS	\$0.00	\$0.00	\$0.00	✓	\$0.00	\$0.00	\$0.00	9 - LESSEE	Rio Tinto Alcan Inc. (0007 37)	LOW	- More specific Job Code identified. Top 3: 3572, 8518, 4092 - Responsibility allocation was overridden	Allocation: It was overridden
☐	3	APPROVED	C	8	8009		09	HIGH PRESSURE WATER BLAST INTERIOR CHUTES ...	\$960.00	\$0.00	\$960.00	✓	\$960.00	\$0.00	\$960.00	9 - LESSEE	Rio Tinto Alcan Inc. (0007 37)	MEDIUM	Responsibility allocation was overridden	Allocation: It was overridden
☐	4	APPROVED	C	1	8018		09	SOLID HEEL DISPOSAL	\$60.00	\$289.48	\$349.48	✓	\$60.00	\$289.48	\$349.48	9 - LESSEE	Rio Tinto Alcan Inc. (0007 37)	MEDIUM	Responsibility allocation was overridden	Allocation: It was overridden
☐	5	APPROVED	C	1	8100		09	LINING INSPECTION	\$300.00	\$0.00	\$300.00	✓	\$300.00	\$0.00	\$300.00	9 - LESSEE	Rio Tinto Alcan Inc. (0007 37)	MEDIUM	Responsibility allocation was overridden	Allocation: It was overridden
☐	6	WAITING ON SHOP	C	30	4450	NX	25	PERFORM LIGHT TEST TO EXTERIOR OF CAR AS P...	\$360.00	\$0.00	\$360.00	✓	\$360.00	\$0.00	\$360.00	9 - LESSEE	Rio Tinto Alcan Inc. (0007 37)	LOW	- More specific Job Code identified. Top 3: 8518, 8020, 8520 - Responsibility allocation was overridden	Allocation: It was overridden
☐	7	WAITING ON SHOP	R	1	4099	PH	09	RUNNING BOARD REINFORCEMENT CLIP	\$0.00	\$8.71	\$8.71	✓	\$0.00	\$8.71	\$8.71	9 - LESSEE	Rio Tinto Alcan Inc. (0007 37)	LOW	- More specific Job Code identified. Top 3: 4404, 4482, 4410 - Responsibility allocation was overridden	Allocation: It was overridden
☐	8	APPROVED	R	2	4410	PH	18	BOLT,HT,5/8 IN DIA OR LESS UNDER 6" LONG (PH...	\$25.20	\$0.40	\$25.60	✓	\$25.20	\$0.40	\$25.60	9 - LESSEE	Rio Tinto Alcan Inc. (0007 37)	MEDIUM	Responsibility allocation was overridden	Allocation: It was overridden
☐	9	WAITING ON SHOP	L	1	4099	PH	09	RUNNING BOARD REINFORCEMENT CLIP	\$0.00	\$8.71	\$8.71	✓	\$0.00	\$8.71	\$8.71	9 - LESSEE	Rio Tinto Alcan Inc. (0007 37)	LOW	- More specific Job Code identified. Top 3: 4404, 4482, 4410 - Responsibility allocation was overridden	Allocation: It was overridden

DAG persists them for async review



Airflow

DAGsCluster ActivityDatasetsSecurityBrowseAdminDocs

00:50 UTC

DAG: brc_process_single

Schedule: NoneNext Run ID: None

DAG Docs

06/22/202612:49:11 AMAll Run TypesAll Run StatesClear Filters

Auto-refresh25

Press **shift** + **/** for Shortcuts

deferredfailedqueuedremovedrestartingrunningscheduledshutdownskippedsuccessup_for_rescheduleup_for_retryupstream_failedno

DAG
brc_process_single

DetailsGraphGanttCodeEvent LogRun DurationTask DurationCalendar

```
graph LR; lookup_context[lookup_context @task] --> route[route @task.branch]; route --> process_revision[process_revision @task]; route --> create_new_brc[create_new_brc @task]; process_revision --> predict_codes[predict_codes @task]; create_new_brc --> predict_codes; predict_codes --> persist_predictions[persist_predictions @task]
```

lookup_context

route

create_new_brc

process_revision

predict_codes

persist_predictions

Bring a human into the loop



> I have an additional use case I would like to prototype. I want to explore using the Astronomer HITL skill to extend the existing `brc_process_single` DAG. After the `predict_codes` task predicts its codes, I would like to prompt the user to choose one before they get persisted

- This is a new feature to prototype, so I'll start with the brainstorming skill before any implementation – same disciplined flow as last time.

- `Skill(superpowers:brainstorming)`
 | Successfully loaded skill

- I'll explore the relevant context first: the existing `brc_process_single` DAG (especially `predict_codes` and the persistence step), and what "Astronomer HITL" actually means against this repo's Airflow version.

Read 2 files (ctrl+o to expand)

- I've read the existing `brc_process_single` DAG. The flow is `lookup_context` → `route` → [`create_new_brc` | `process_revision`] → `predict_codes` → `persist_predictions`, where `predict_codes` returns top-K rows `{brc_detail_id, prediction_type, rank, label, probability}` and `persist_predictions` writes **all ranks** to `brc.brc_detail_predictions`.

One thing I need to verify before anything else: **Astronomer/Airflow HITL is an Airflow 3.x feature**, but this repo's image is pinned to `apache/airflow:2.10.4`. That's a potential hard blocker, so let me confirm the actual version requirement from Astronomer's docs rather than assume.

Bring a human into the loop



- I have enough to shape the design. Here it is in four sections – react to any of them.

Design: HITL job-code selection in `brc_process_single`

1. Environment (isolated Airflow 3.1)

A new self-contained `brc/pipeline/airflow-hitl/` with its own `docker-compose.yml` on `apache/airflow:3.1.x`, joining the existing `aitx-network`. It **reuses** the shared dev **SQL Server** and the **model prediction service** (only Airflow itself is new) – so `predict_codes` and the `brc_details` update hit the same DB/model as today. Crucially, HITL operators are **deferrable**, so this compose must include a **Triggerer** component (the 2.10 compose doesn't), alongside the Airflow 3 `api-server/scheduler/dag-processor`. The 2.10.4 setup and all its DAGs are untouched.

2. DAG flow & the HITL gate

A ported copy `brc_process_single_hitl` keeping the real shape, with the gate inserted between predict and persist:

```
lookup_context → route → [create_new_brc | process_revision]
  → predict_codes (unchanged: top-K job_code + why_made per detail)
  → build_choice_inputs (reshape: one {brc_detail_id, options:[top-K job codes], narrative} per detail that got a job_code prediction)
  → choose_job_code (HITLOperator, dynamic-mapped .expand() – one form per detail line; options = that line's predicted job codes)
  → apply_choice (per detail: UPDATE brc.brc_details with the chosen code)
```

The human answers each mapped form in the Airflow 3 UI **"Required Actions"** inbox; the chosen option returns via XCom to `apply_choice`.

3. Data & persistence

- `apply_choice` (mapped per detail) runs `UPDATE brc.brc_details SET legacy_applied_job_code = <chosen>, job_code_id = (SELECT job_code_id FROM brc.brc_job_codes WHERE job_code = <chosen>) WHERE brc_detail_id = ?`.
- Details **without a narrative/prediction** get no form and are skipped (matches today's predict behavior).
- **Scope cuts (YAGNI):** `why_made` is **out of scope** (job_code only); the original `persist_predictions` (writing top-K to `brc_detail_predictions`) is **replaced** by `apply_choice` – the chosen code applied to the line is the prototype's outcome.
- **No timeout** for the prototype (the form waits for a human). I'll note `HITLOperator`'s `defaults/timeout` knob as a documented future toggle (e.g., `auto-pick rank-1`).

Line Items

Export

All

Needs Review

Waiting on Shop

Approved

Rejected

All (46)

Owner (35)

Lessee (11)

Search

Item #

↑↓

Status

↑↓

Loc

Qty

Job Code

↑↓

QA

WMC

Description

Labor

Material

Total

↑↓

CB

CB Labor

CB Material

CB Total

Responsible

↑↓

CB Location

☐	1	APPROVED	C	1	9825	Choose	ZOG	TQ	TANK QUALIFICATION - EP 30267	\$0.00	\$0.00	\$0.00	—	—	—	—	1 - OWNER	—	
☐	2	APPROVED	C	1	8518	Choose	09		INBOUND INSPECTION - TANK VISUAL AND SAFET...	\$125.00	\$0.00	\$125.00	—	—	—	—	1 - OWNER	—	
☐	3	APPROVED	C	1	9750	Choose	01	TQ	INSULATION - IN	—	—	—	—	—	—	—	1 - OWNER	—	
☐	4	APPROVED	A	2	9815	Choose		TQ	NON-DESTRUCTIVE	—	—	—	—	—	—	—	1 - OWNER	—	
☐	5	APPROVED	B	2	9815	Choose		TQ	NON-DESTRUCTIVE	—	—	—	—	—	—	—	1 - OWNER	—	
☐	6	APPROVED	C	1	9817	Choose	ZYF	TQ	TANK QUAL THICK	—	—	—	—	—	—	—	1 - OWNER	—	
☐	7	APPROVED	C	1	9818	Choose	ZYT	TQ	TANK QUAL THICK	—	—	—	—	—	—	—	1 - OWNER	—	
☐	8	APPROVED	C	1	9821	Choose		TQ	SERVICE EQUIPM	—	—	—	—	—	—	—	1 - OWNER	—	
☐	9	APPROVED	C	1	9000	Choose	02	TQ	GAUGE DEVICE	—	—	—	11	✓	\$93.75	\$44.36	\$138.11	9 - LESSEE	Chevron Phillips Chemical
☐	10	APPROVED	C	1	9002	Choose	02	25	GAUGE ROD - B	—	—	—	32	✓	\$31.20	\$437.12	\$468.32	9 - LESSEE	Chevron Phillips Chemical
☐	11	APPROVED	C	3	9430	Choose	02	TQ	ANGLE VALVE - 2 INCH STEEL, VITON GFS	\$150.00	\$2,178.80	\$2,328.80	✓	\$150.00	\$2,178.80	\$2,328.80	9 - LESSEE	Chevron Phillips Chemical	
☐	12	APPROVED	C	3	9442	Choose		TQ	GASKET, ANGLE VALVE - VSP 0320, KLINGER C-450...	\$0.00	\$9.42	\$9.42	✓	\$0.00	\$9.42	\$9.42	9 - LESSEE	Chevron Phillips Chemical	
☐	13	APPROVED	C	1	9555	Choose		TQ	VALVE, STAINLESS STEEL, SAMPLE LINE - VITON GFS	\$62.50	\$281.32	\$343.82	—	—	—	—	1 - OWNER	—	

Choose Job Code

SERVICE EQUIPMENT INSPECT AND TEST - current 9821

MODEL SUGGESTIONS

☒

4404

Ladder repair

92%

☐

4592

Grab iron replacement

71%

☐

4412

Sill step replacement

45%

☐

Keep original (9821)

Cancel

Update

Choose Job Code



SERVICE EQUIPMENT INSPECT AND TEST - current 9821

MODEL SUGGESTIONS

- ☒ 4404 Ladder repair 92%
- ☐ 4592 Grab iron replacement 71%
- ☐ 4412 Sill step replacement 45%
- ☐ Keep original (9821)

Cancel

Update

Key Takeaways for ML



- Understanding your User is the hard part
- Applicable to multiple ML technologies (classical ML, LLMs)
- Most time spent refining requirements and planning
- Parts of the process that can't be TDD are delegated to a human
- Reviews occur in real-time based off rules or thresholds



Questions?

kyle@txidigital.com,
@KyleMMCC

Instructions



You do not really need to use this template for all your slides. As long as you use the title/cover slide, that is enough. You can use your own template for the rest of your presentation.

But if you want to use this, that is great!